Teaching a robot arm with my own hand

I filmed a pick-and-place task 54 times on my phone, turned my hand into robot demonstrations in the LIBERO simulator and fine-tuned SmolVLA on them. No robot data was used.



Left: one of my phone demos. Right: the Panda arm replaying it in simulation.

96%

success on the LIBERO task, trained only on my videos

Yash Joshi
MEng Robotics and AI, UCL
Humanoid internship challenge
github.com/yash-joshi5379/Real2Sim-Task

THE CHALLENGE

Use data I recorded myself to drive a robot in simulation



The simulator task: LIBERO-Object #4, "pick up the ketchup and place it in the basket"



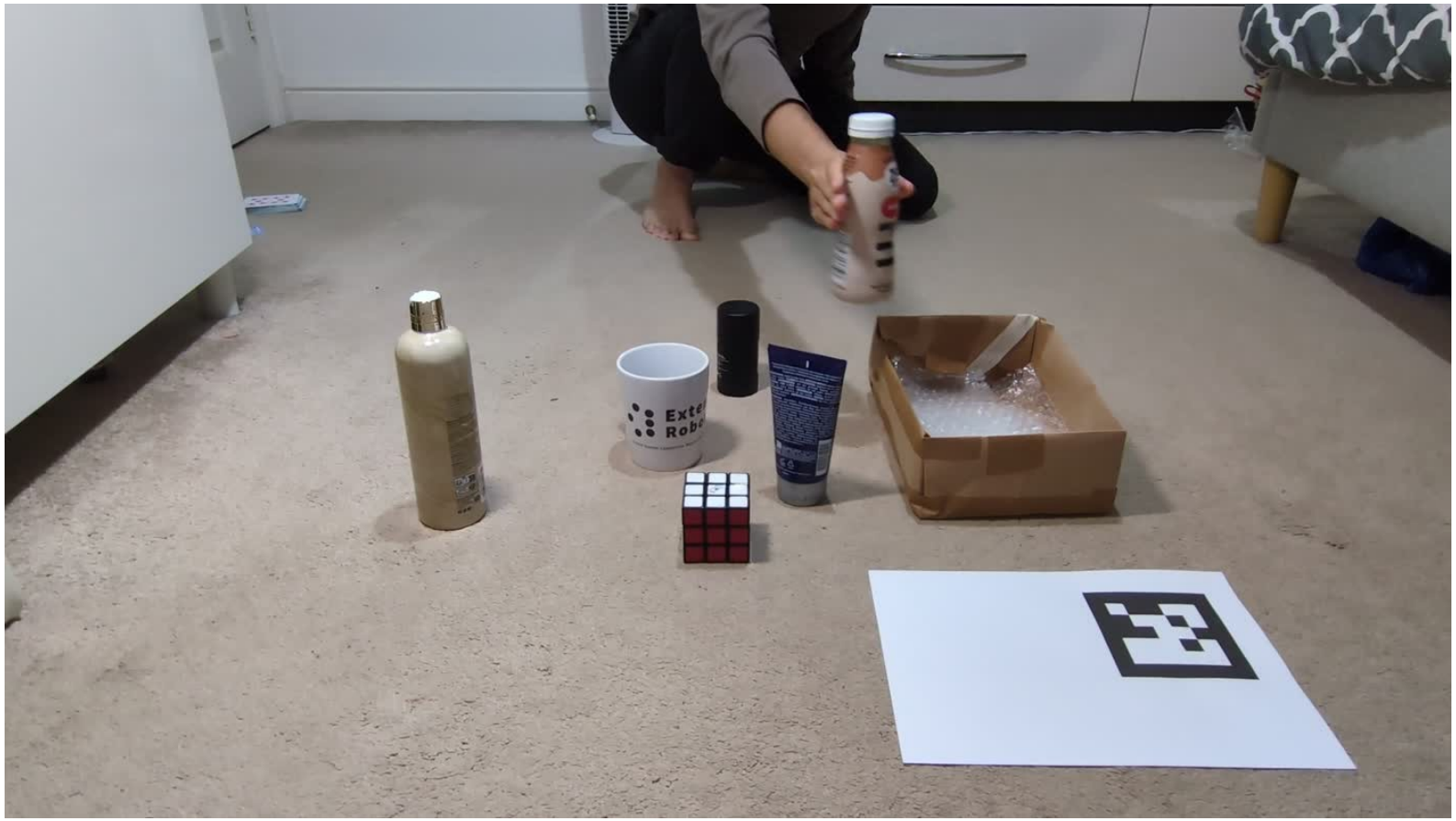
My version at home: a bottle for the ketchup, a box for the basket

- Why this task: I had a bottle and a box at home that match the ketchup and basket, and LIBERO tests it on 50 fixed layouts, so results are comparable
- Plan: record, extract my hand's 3-D path, retarget it onto the robot, train a VLA, evaluate the standard LIBERO way
- Reference: the same model trained on LIBERO's 45 robot demos of this task, so any gap comes from the data, not the model

1 Record | **2 Extract** | **3 Retarget** | **4 Train** | **5 Evaluate & iterate**

STEP 1 · RECORD

54 demos with a phone and a printed marker

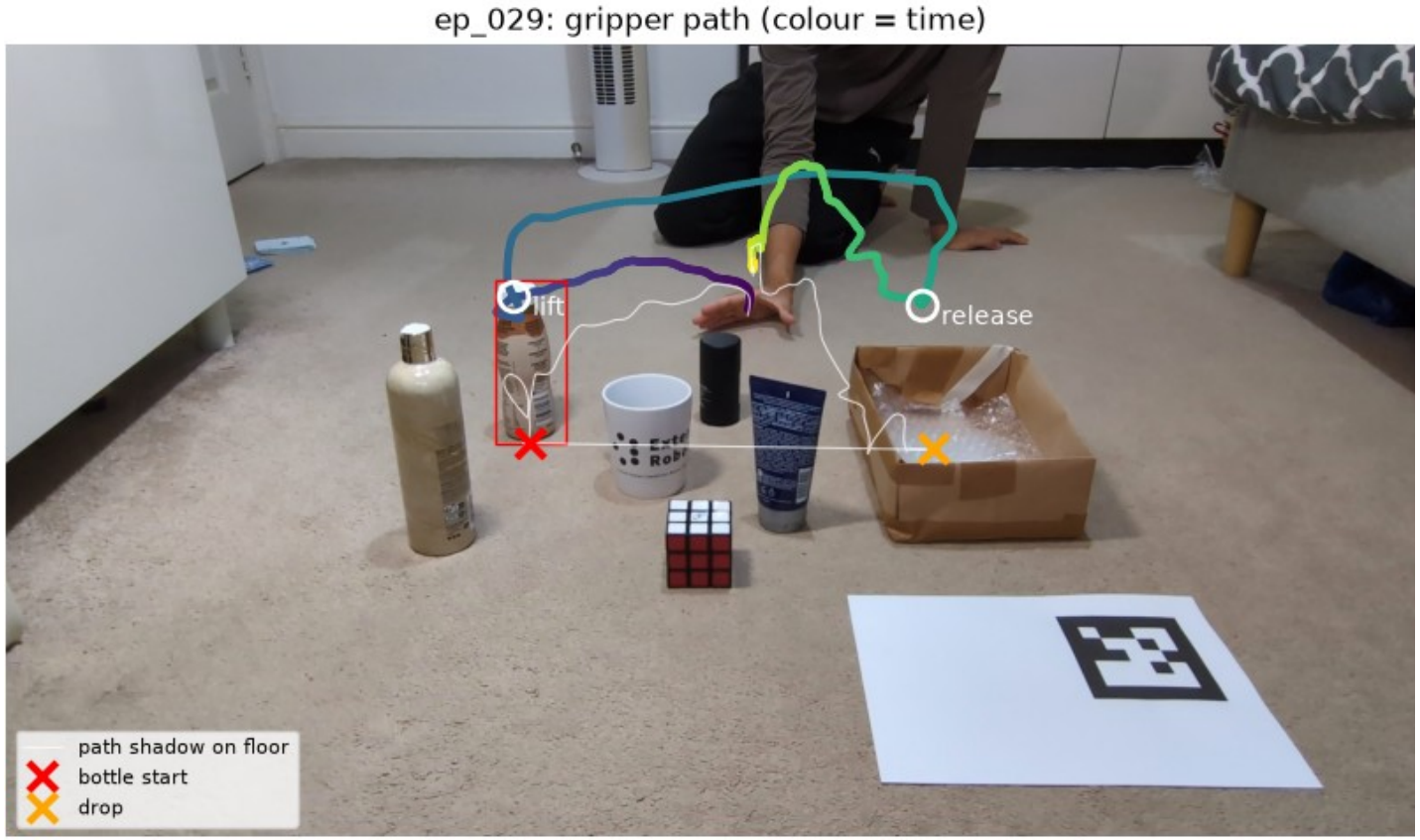


A frame from one of the 54 clips, mid-carry.

- Phone fixed on a stand: the camera never moves, so the marker calibrates a whole clip at once
- ArUco marker (100 mm): its known size gives metric scale and the floor plane
- Printed on A4: the sheet's known edges give the focal length, so no separate calibration
- 54 demos: about as many as LIBERO's own 45 for this task, so the comparison is fair
- Distractors around the bottle, like the cluttered LIBERO scene

STEP 2 · EXTRACT

From video to a 3-D path of my hand



One demo: the tracked grasp point over time (purple → yellow) and its shadow on the floor.

- MediaPipe hands: runs on a laptop CPU and gives a metric 3-D hand model
- Grasp point = thumb-index midpoint, where a parallel gripper's fingers close
- Bottle start and drop from the first vs last frame: it's the only thing that moved, so no object detector is needed
- Carry depth from geometry (hand ray meets the vertical plane between bottle and box), because single-image hand depth was up to 40% off
- 51/54 clean; 3 neck grasps kept, as their paths are still valid

STEP 3 • RETARGET

The robot replays every demo in LIBERO



The caption under the robot shows when it follows my hand and when it acts on its own.

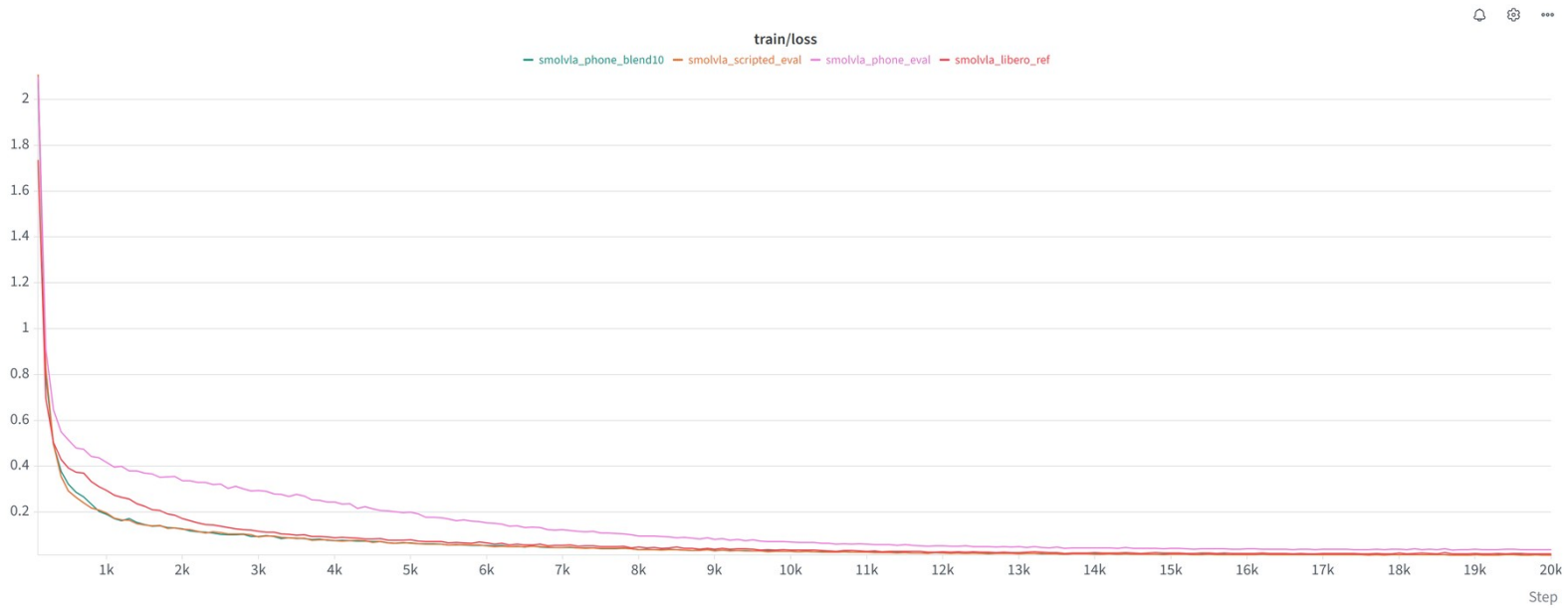
- One transform for all demos, fitted on where my hand let go (the bottle tipped ~6 cm further after release)
- Heights rescaled: my bottle is 19 cm, the sim ketchup 14.6 cm
- The robot follows my reach and carry (70% of steps) but does the contact itself, because a phone can't see finger contact and copied grasps slipped
- Played 1.3× faster to fit LIBERO's 280-step limit; 54/54 succeed

Why replay in the simulator instead of training on the video

A VLA learns from robot camera images paired with robot actions. Replaying with LIBERO's own controller turns my motion into exactly that, in the action space LIBERO policies use.

STEP 4 · TRAIN

Same model, same settings as for LIBERO's own data



Training loss in Weights & Biases (B, C2, D, G). D's loss stays highest: its 54 paths are all different.

- Same format as LeRobot's LIBERO data, so my runs and the reference share identical code
- SmolVLA: pretrained, which matters with 54 demos, and small enough (~450M) for one 16 GB GPU
- Same recipe every run (batch 32, 20k steps, seed 1000), so only the data changes
- 20k steps: the loss has flattened, and D-40k gained nothing
- Scored with lerobot-eval on the 50 standard layouts, the way LIBERO results are reported

STEP 5 · EVALUATE AND ITERATE

12 runs in the order I ran them, one change each

Run	What changed	Success	What it told me
A	My demos as extracted	**18%**	Far below the reference: something is wrong
B	LIBERO's own robot demos (reference)	**86%**	The target
A2	Bottle at random spots of the spawn area, ×4	**22%**	Spreading the bottle out doesn't help
C	Scripted motion instead of mine (control)	**0%**	Even perfect motion fails: not my hand
D	Bottle exactly where evaluation puts it	**50%**	Placement was the first problem
C2	Scripted motion + that placement (control)	**98%**	The pipeline works; my paths cost the rest
D4	D, each demo in 4 layouts	**24%**	More copies of the same motions: no
D-40k	D, trained twice as long	**48%**	Not under-training
E	D with one fixed timing	**30%**	Not the timing
F	D's data, ACT instead of SmolVLA	**18%**	Pretraining matters with 54 demos
G	**D with the path shapes averaged**	**96%**	**Consistent paths fix it**
H	D with tracking jitter filtered out	**52%**	Not measurement noise

Success = ketchup in the basket, over LIBERO's 50 test layouts (final checkpoints).

DIAGNOSIS

Where the 50% policy went wrong



Hovers, never grasps

Knocks the ketchup over

Drops it beside the basket

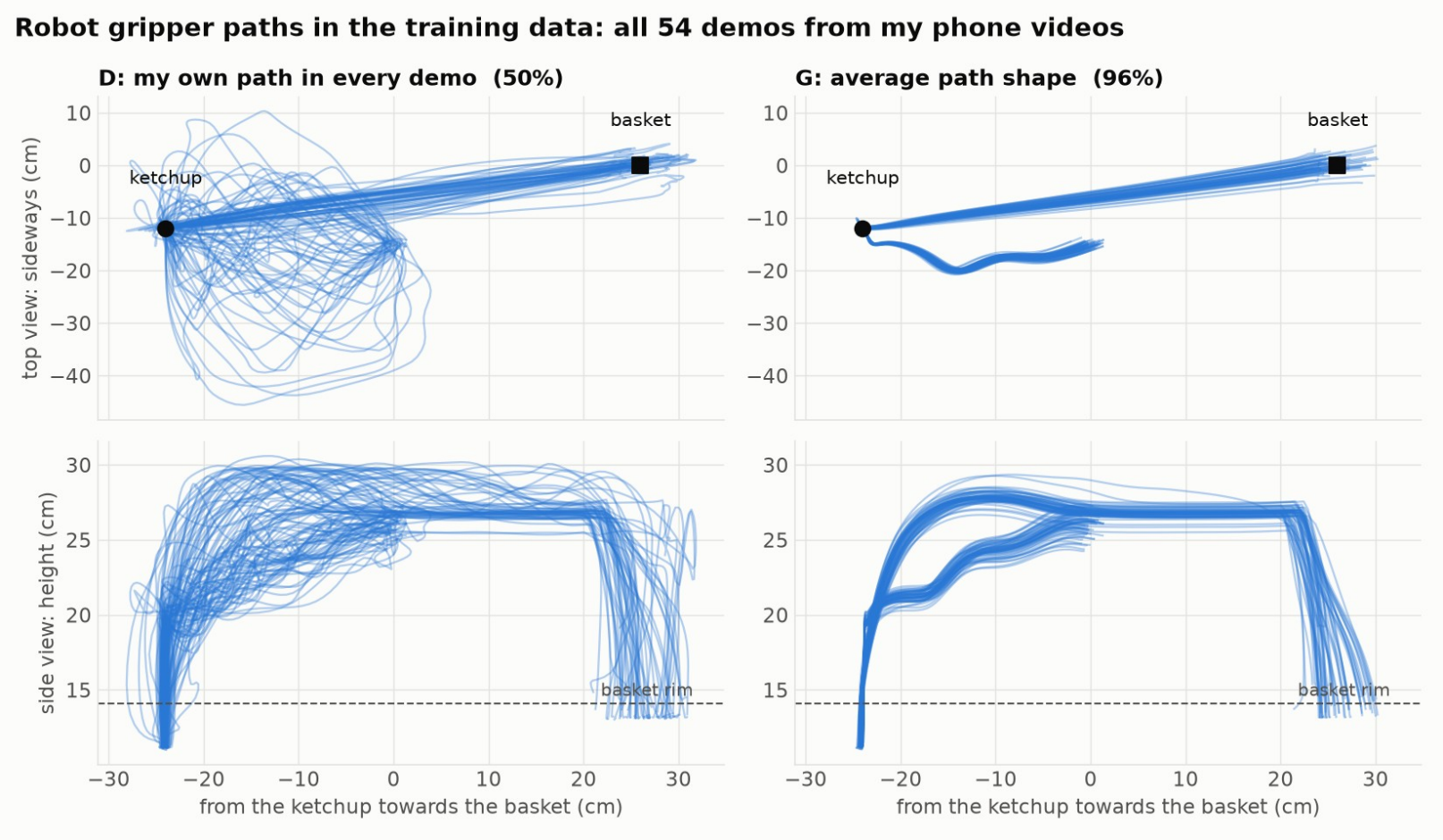
- Grasps are hesitant: the policy has learned a blur of 54 different paths
- The controls ruled out the pipeline (C2: 98%)
- E, H and D-40k ruled out timing, tracking noise and training length
- What's left: the paths themselves vary too much between demos

A mistake I corrected

I first blamed C's 0% on the slow approach in the scripted motion. C2 has the same approach and scores 98%, so C was failing because of where the bottle was.

THE FIX

Averaging the shape of my 54 paths



- Each path = straight line from its start to its end + how it bends away from that line
- The bends are averaged over all 54 demos and put back on each demo's own line
- Every demo keeps its own start, grasp, release and timing: it's still my motion, just consistent
- Why this fix: more copies (D4) and a spread-out bottle (A2) didn't help, and C2 showed consistent paths reach 98%
- Caveat: it works this well because the test always puts the ketchup in the same place

RESULT

96% from phone videos, above LIBERO's own robot demos



The trained policy (run G) in three of LIBERO's test layouts, real time.

96%

G: my demos, averaged paths

86%

B: LIBERO's robot demos

50%

D: my demos, raw paths

Same model, same training, same test

Only the training data differs between G, B and D, so the comparison is like for like. Intervals: G 87–99%, B 74–93% (50 episodes each).

TAKEAWAYS

What is different about this approach

My data drives the robot

70% of every training episode is my own hand motion. The robot only adds the last centimetres of the grasp and release.

A fair comparison

Same controller, data format, model, training and test as LIBERO's own demos, so 96% vs 86% is like for like.

A diagnosis, not just a score

Controls and one-change runs showed why the first version failed and what fixed it.

Cheap to repeat

A phone, a printed marker and a laptop for the data; one command per experiment.

ABOUT ME

How this matches what the role asks for

The role asks for	Evidence from this challenge and my work
Master's or PhD in CS / ML / robotics	MEng Robotics and Artificial Intelligence, UCL (penultimate year, graduating June 2028)
ML foundations, Python, PyTorch	Pipeline in Python; SmolVLA and ACT fine-tuned in PyTorch through LeRobot
Interest in RL, world models, VLAs	VLA post-training on human data, closing the gap between human and robot data; RL and world models are my next steps
Running experiments rigorously	12 runs, one change each, controls, 95% intervals, and an explanation I retracted when the data disagreed
Ownership, iterating with guidance	Chose the task, recorded the data, built every stage; led a 5-person VLA project with Extend Robotics
Problem solving, attention to detail	Found that the test always uses one ketchup position; fixed depth errors, arm lag and grasp slips
Fast learner, research pace	New to LIBERO, retargeting and hand tracking; first recording to 96% in about two weeks, on a laptop and a shared GPU

NEXT STEPS

With more time, data and compute

More seeds and tasks

One seed and 50 episodes per run means gaps under ~15 points are noise; 3 seeds and more LIBERO tasks would settle them.

Beyond one position

The test keeps the ketchup in one spot, real use won't: demos with the bottle moved, blended only part-way.

RL on top of the VLA

Imitation can't recover from states it never saw; RL in LIBERO starting from G can learn to.

World model

The bottleneck was few, inconsistent demos; a video model trained on my clips could generate more.

Better hand tracking

30% of each episode is scripted because a phone can't see finger contact; a 3-D hand mesh (HaMeR, WiLoR) could.

Faster inference

A real robot has a latency budget; profiling and quantising SmolVLA would show what fits.

Code, results and instructions: **github.com/yash-joshi5379/Real2Sim-Task**